

Algorithms and Data Structures I.

Practice 6

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

March 19, 2024

Content in this Practice I

- 1 Review last week -
Linked lists - One-way lists
 - Linked lists
 - Simple one-way list (S1L)
 - One-way lists with header node (H1L)
- 2 Linked lists - Two-way (doubly)
 - Simple two-way (doubly) linked list (S2L)
 - Cyclic two-way list (C2L) with header
 - Problem: Sorted list filtered to even and odd sorted
- 3 Union of two C2L

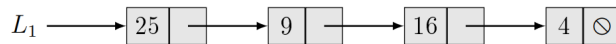
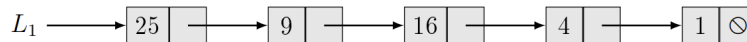
Review last week - Linked lists - One-way lists

Linked lists

- A **linked list** is a data structure in which the objects are arranged in a linear order. Unlike an array, however, in which the linear order is determined by the array indices, the order in a linked list is determined by a pointer in each object. *Source: Introduction to Algorithms by Cormen et. al.*
- Types:
 - Simple one-way list (S1L)
 - One-way lists with header node (H1L)
 - Cyclic one-way list (C1L)
 - Simple two-way or doubly linked list (S2L)
 - Cyclic two-way list (C2L) with header

Simple one-way list (S1L)

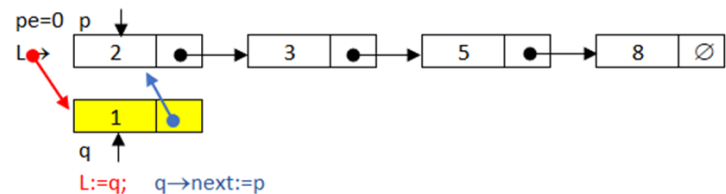
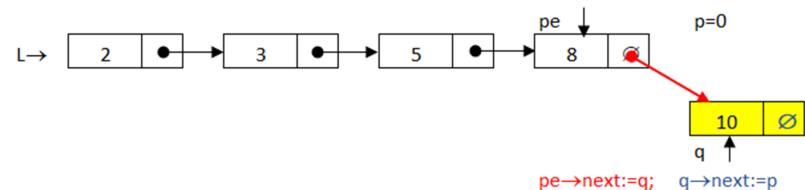
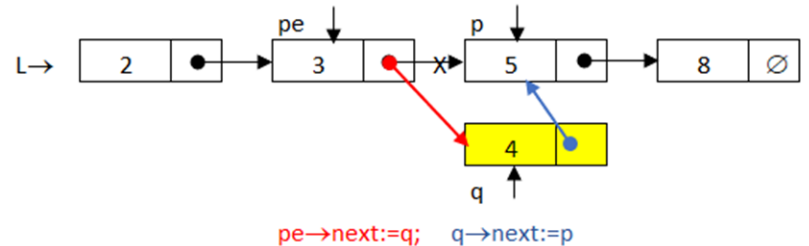
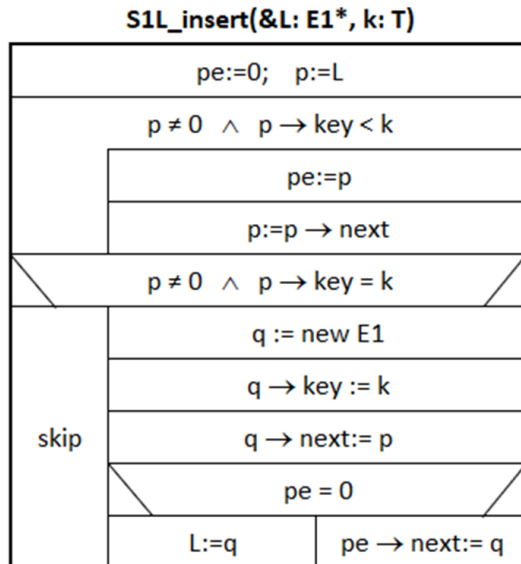
$L_1 = \emptyset$



- Each list element has its general data, and a pointer pointing to the next element in the list. If it's the last element, the pointer is NULL.
- L points to the first element, but L is NOT an element in itself, nor is it a type of $E1$.

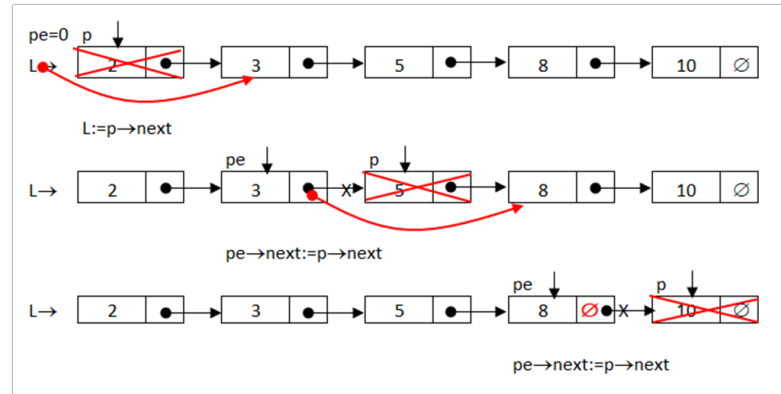
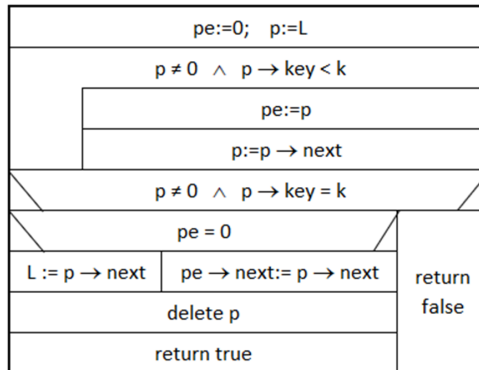
E1
$+key : \mathcal{T}$
$\dots$ // satellite data may come here
$+next : E1^*$
$+E1() \{ next := \emptyset \}$

Simple one-way list (S1L) - Insertion



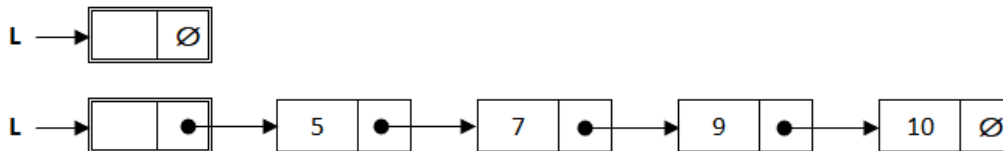
Simple one-way list (S1L) - Deletion

S1L_delete(&L:E1*, k: T):B



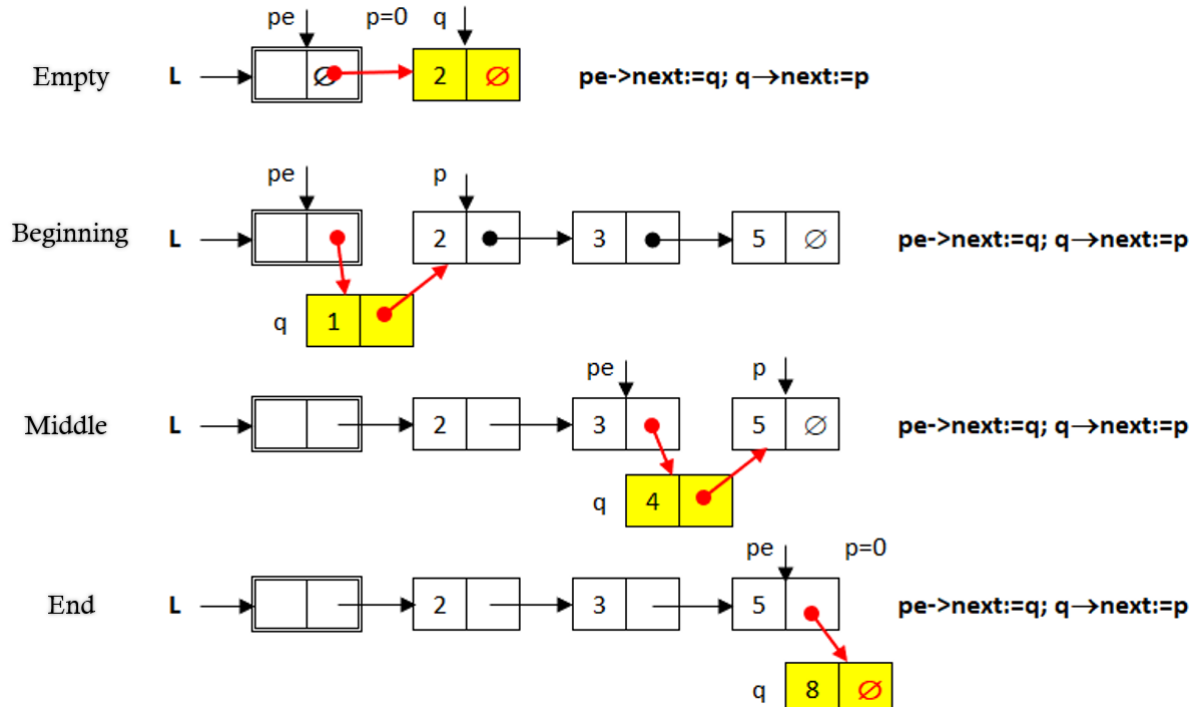
One-way lists with header node (H1L)

- Header element:
 - As we've seen these algorithms need to handle the beginning of the list differently from the other cases since `pe` is null.
 - To solve this issue, and avoid unnecessary branches we introduce a header element.
 - This way the list will always have an E1 element, sort of a placeholder!
 - This also means that we however don't store data in it.

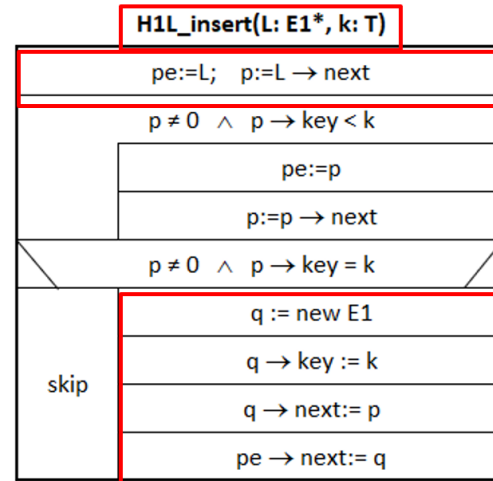
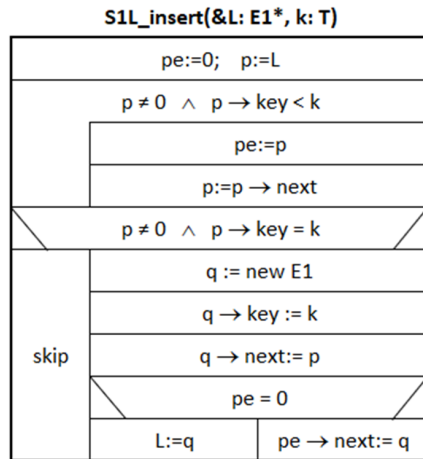


- Searching will be slightly different:
 - We start from the first element after the head

One-way lists with header node (H1L) - Insertion

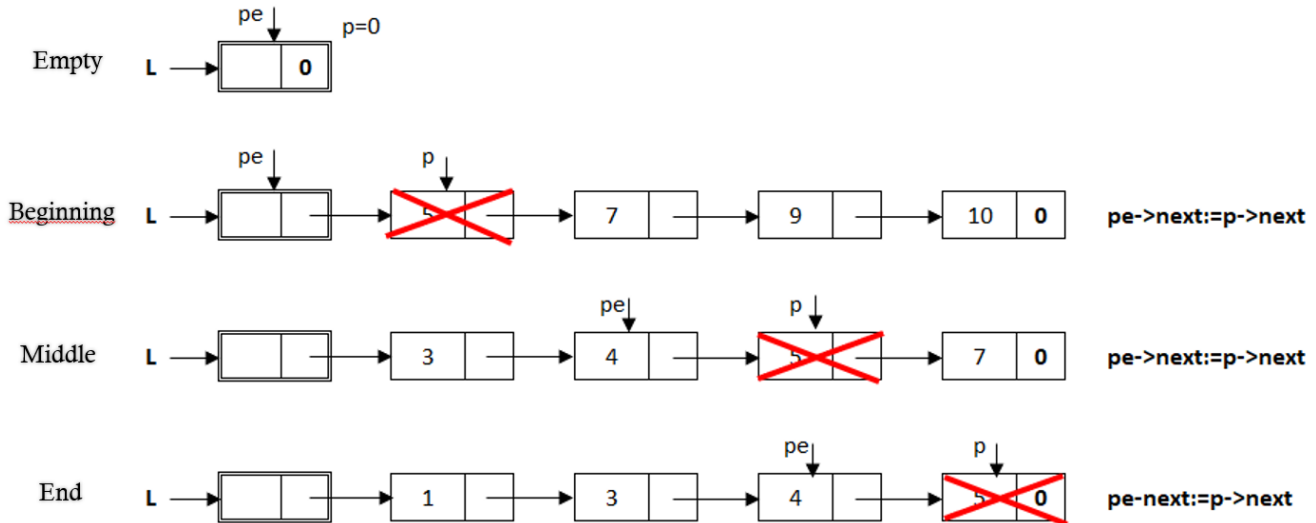


One-way lists with header node (H1L) - Insertion

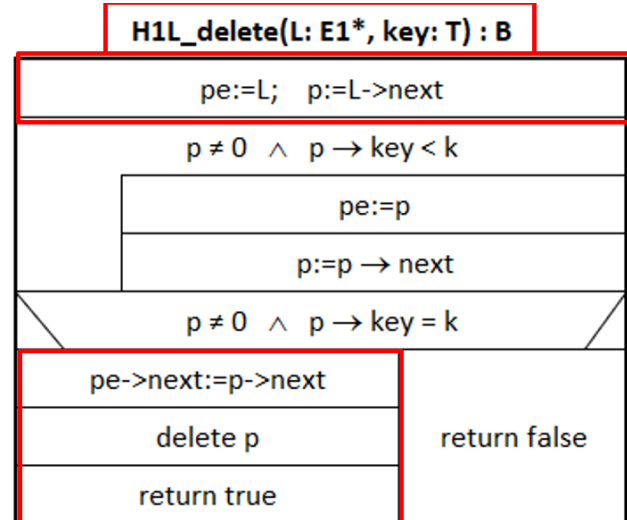
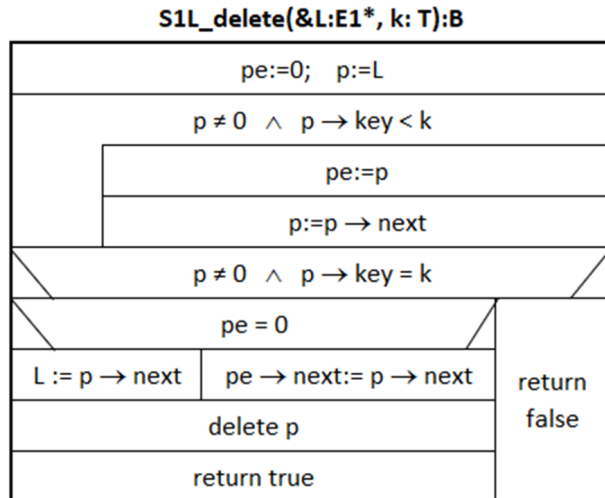


- Differences:
 - L is not passed by reference.
 - Start: pe is the header, p is the first element (can be null)
 - No branch is needed since pe is never null.

One-way lists with header node (H1L) - Deletion



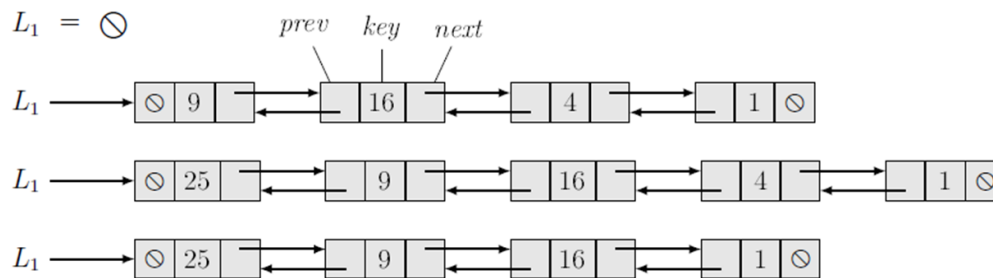
One-way lists with header node (H1L) - Deletion



- Differences:
 - L is not passed by reference.
 - Start: pe is the header, p is the first element (can be null)
 - No branch is needed since pe is never null.

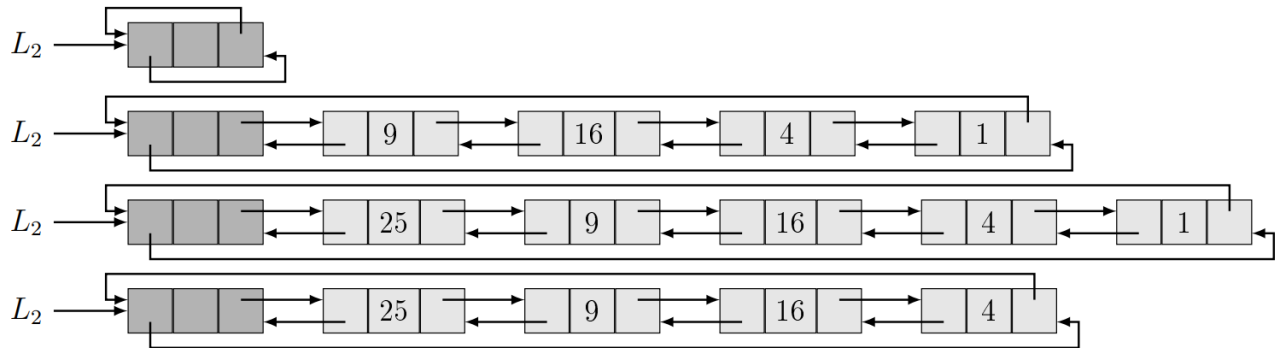
Linked lists - Two-way (doubly)

Simple two-way (doubly) linked list (S2L)



- Advantage: Easy to ‘maneuver’.
- Handling the inner, last, and first elements still, once again requires different handling:
- An element inside the list has prev and next neighbors, which means we must handle 4 pointers.
- The first element has no previous neighbor, meanwhile the last element has no next.
- Inserting into an empty list means that the element to be inserted won’t have any neighbors.

Cyclic two-way list (C2L) with header



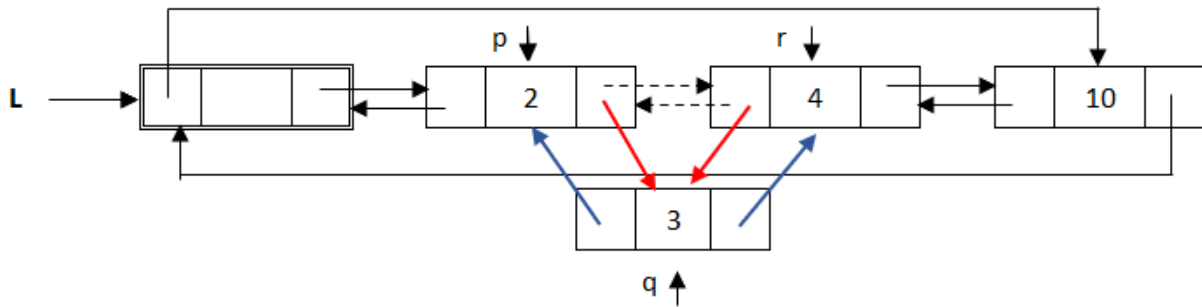
- Each element will contain the prev and the next pointer.
- The 'cyclic' aspect refers to the following:
 - The prev value of the head element always points to the last element of the list
 - The next pointer of the last element in the list therefore points to the first element of the list

Cyclic two-way list (C2L) functions, E2 Node type

E2
$+prev, next : \mathbf{E2}^*$ // refer to the previous and next neighbour or be this
$+key : \mathcal{T}$
$+ E2() \{ prev := next := \mathbf{this} \}$

$\text{precede}(q, r : E2^*)$ // (*q) will precede (*r) $p := r \rightarrow prev$ $q \rightarrow prev := p ; q \rightarrow next := r$ $p \rightarrow next := r \rightarrow prev := q$	$\text{follow}(p, q : E2^*)$ // (*q) will follow (*p) $r := p \rightarrow next$ $q \rightarrow prev := p ; q \rightarrow next := r$ $p \rightarrow next := r \rightarrow prev := q$
$\text{unlink}(q : E2^*)$ // remove (*q) $p := q \rightarrow prev ; r := q \rightarrow next$ $p \rightarrow next := r ; r \rightarrow prev := p$ $q \rightarrow prev := q \rightarrow next := q$	

Cyclic two-way list (C2L) functions - precedence



$\text{precede}(q, r : E2^*)$

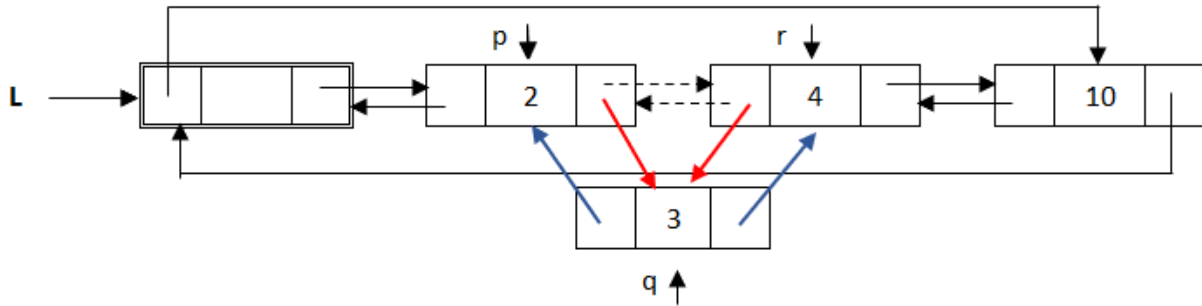
// (*q) will precede (*r)

$p := r \rightarrow \text{prev}$

$q \rightarrow \text{prev} := p ; q \rightarrow \text{next} := r$

$p \rightarrow \text{next} := r \rightarrow \text{prev} := q$

Cyclic two-way list (C2L) functions - follow



$\text{follow}(p, q : E2^*)$

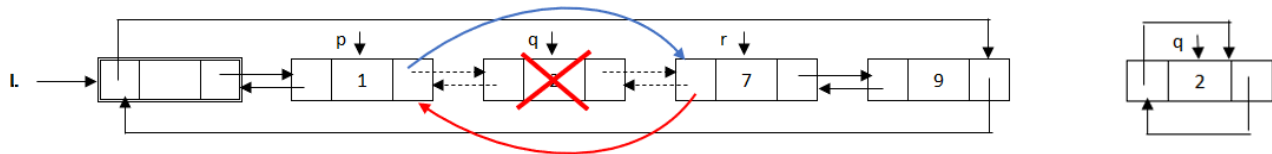
// (*q) will follow (*p)

$r := p \rightarrow next$

$q \rightarrow prev := p ; q \rightarrow next := r$

$p \rightarrow next := r \rightarrow prev := q$

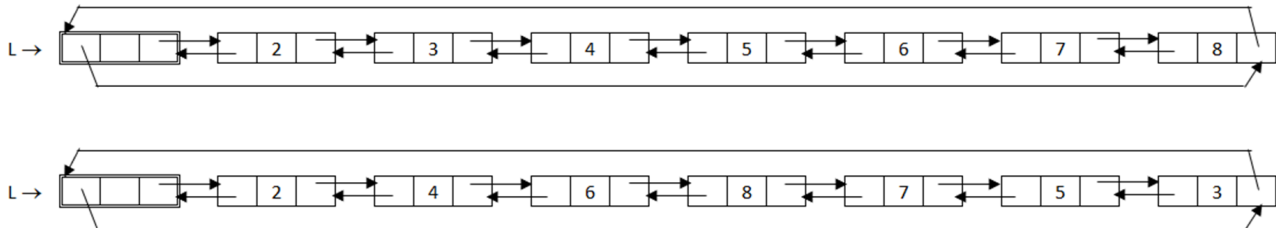
Cyclic two-way list (C2L) functions - unlink



$\text{unlink}(q : E2^*)$

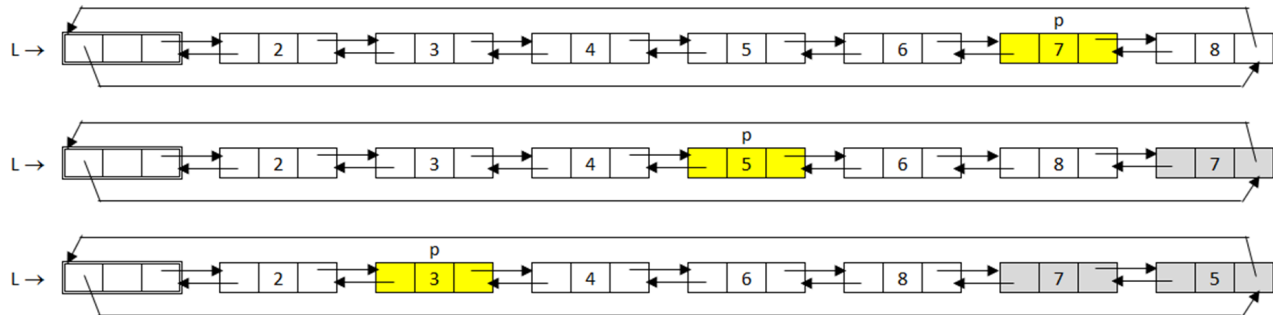
$// \text{ remove } (*q)$
$p := q \rightarrow \text{prev} ; r := q \rightarrow \text{next}$
$p \rightarrow \text{next} := r ; r \rightarrow \text{prev} := p$
$q \rightarrow \text{prev} := q \rightarrow \text{next} := q$

Sorted list filtered to even and odd sorted



- Let L point to an ordered C2L list containing natural numbers.
- Let's reorder the list so that at the 'front' of the list there are only even numbers, and at the 'end' there are only odd numbers.
- Note: Both 'parts' of the list should remain ordered.

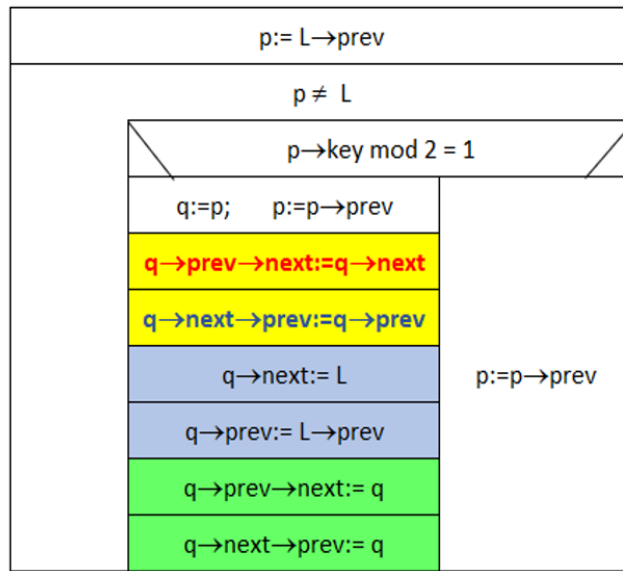
Sorted list filtered to even and odd sorted



- Iterate on the list backward with p .
- If p is odd, we link it before the header (=end of the list)

Sorted list filtered to even and odd sorted

ReorderC2L(L:E2*)



We only have to do anything if p is odd

Yellow: Linking out the odd element

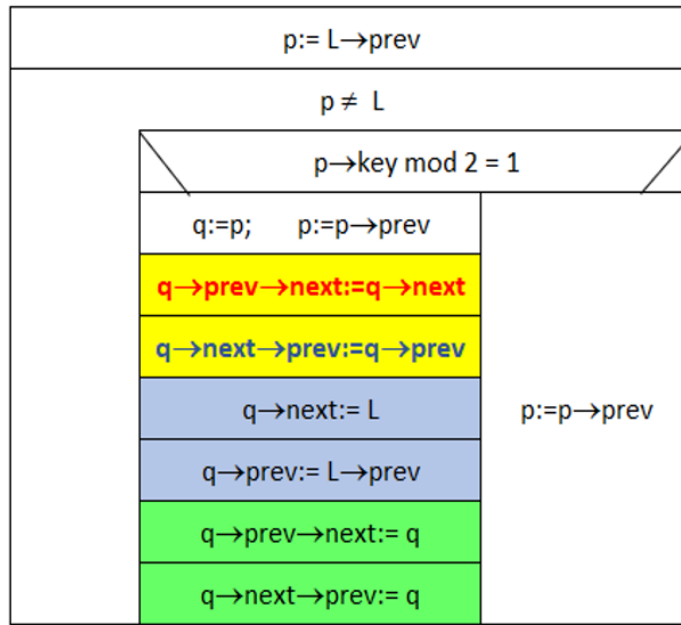
Blue: Inserting q before the header.

Green: Setting the neighbours' pointers

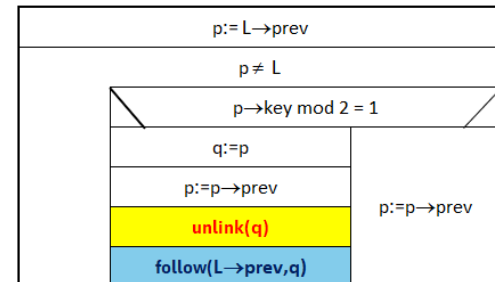
33

Sorted list filtered to even and odd sorted - simplified

ReorderC2L(L:E2*)



Simplified ReorderC2L(L:E2*)

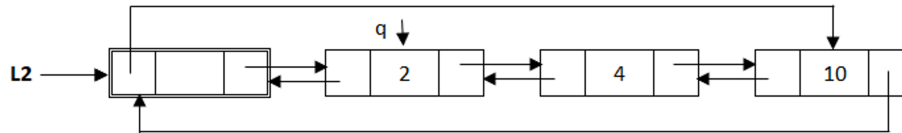
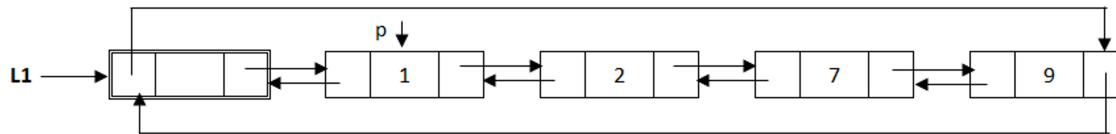


Union of two C2L

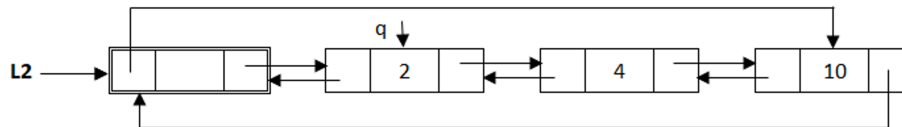
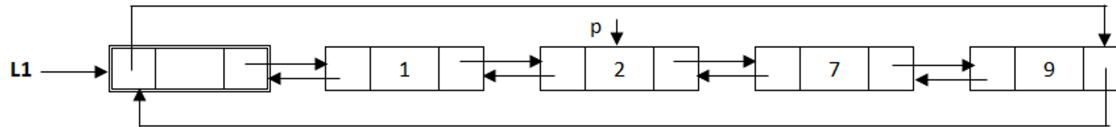
Union of two C2L

- Combining two lists is a frequent task to solve, and not just for job interviews. The sorted nature of these lists plays an essential role in the efficiency of these algorithms. Since we are no longer working with arrays, inserting an element does not require shifting others.
- Usually we work with lists containing a header (H1L, C2L).
- An example: Let L1, and L2 each point to different sorted C2Ls. Let the keys also be unique (the list therefore can be considered as a set). Let the output be L1 and the list it corresponds to be the union of the two sets.
- Idea: Each list has a pointer iterating through its elements. Then, we have 3 cases

Union of two C2L - case 1

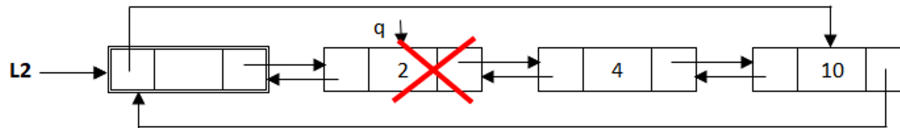
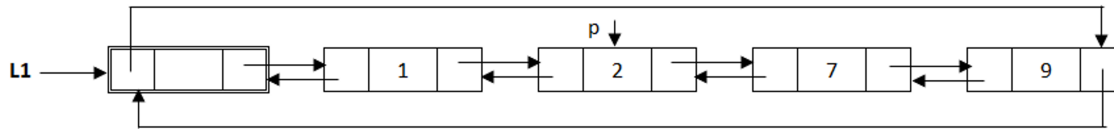


P keeps going, but q remains in place

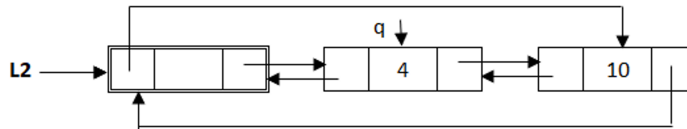
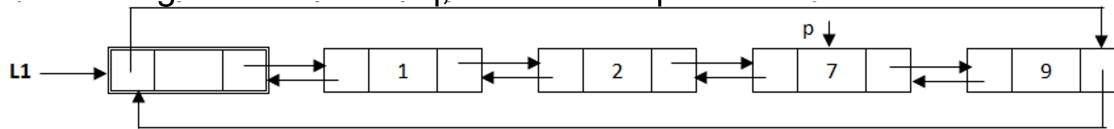


- (1) The current element in L1 is smaller than the element in L2, we step with the pointer in L1, but remain stationary in L2.

Union of two C2L - case 2

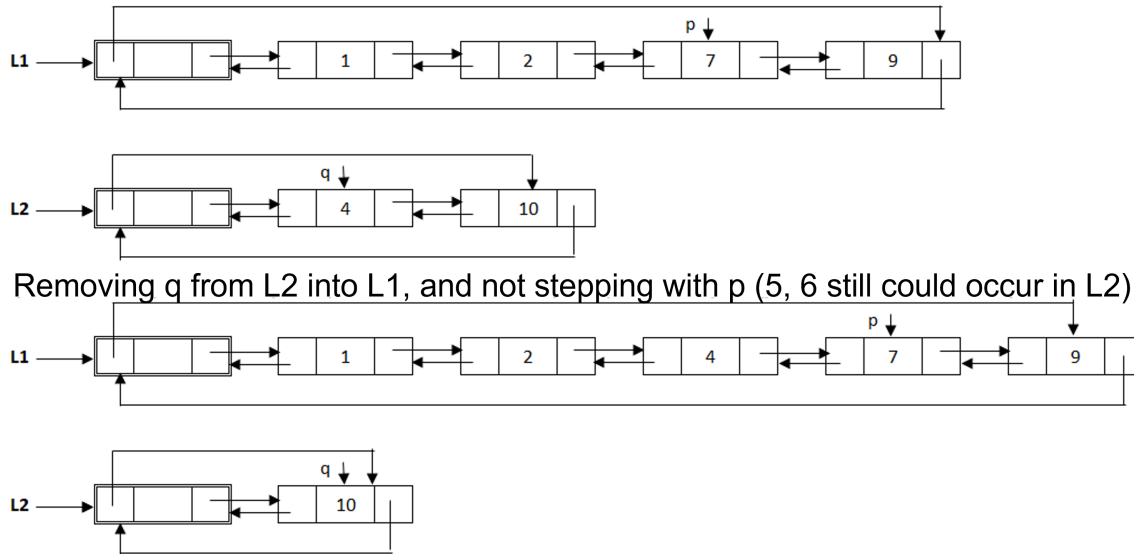


Deleting the element at q, and then step with both



- (2) The value of the current element in L1 equals to the current element in L2. We remove the element from L2 and move with both pointers.

Union of two C2L - case 3



- (3) L1 has a larger element at p , than the element at q . Then we insert the element at q into L1 by removing it from L2, and step with q , while not moving with p .

Union of two C2L - Reaching the end of a list

- If $p=L1$ then we reached end of L1, otherwise if $q=L2$, then the end of L2. (It could occur at the same time in cases where we step with both.)
- Reaching the end of L2 means we are done, there are no more elements to potentially insert into L1. Otherwise, if we reach the end of L1, that is equivalent to saying that there are still elements in L2 that are not present in L1. Meaning we need to copy the remaining L2 into L1. This can be done one by one, but in constant time too!
- Note: At any point in the algorithm the following is true: $q=L2 \rightarrow \text{next}$. This means that we don't need the q variable when we implement it, but for readability, we will still use it for now.

Union of two C2L

Unio(L1: E2*, L2: E2*)

p := L1 → next; q := L2 → next		
p ≠ L1 ∧ q ≠ L2		
p → key < q → key	p → key = q → key	p → key > q → key
p := p → next	s := q	s := q
	q := q → next	q := q → next
	L2 → next := q	L2 → next := q
	q → prev := L2	q → prev := L2
	delete s	s → prev := p → prev
	p := p → next	s → next := p
		s → prev → next := s
q ≠ L2		
e := L2 → next; u := L2 → prev	skip	
e → prev := L1 → prev; L1 → prev → next := e		
L1 → prev := u; u → next := L1		
L2 → next := L2 → prev := L2		

- Blue:
 - Current element in L1 is smaller than the current element in L2: Just step with p.
- Yellow:
 - Elements are equal. This means that L1 already contains that element, so we just have to remove the element at q from L2, and set it to the next element.
- Green:
 - Found an element which is not present in L1, meaning we have to insert it from L2 to L1. This is the same as the previous except this time we not only remove q from L2, but also insert it into L1, and not step with p.
- Red:
 - Append the remaining elements from L2 to L1.
- Note:
 - The order of some instructions is essential in order to work properly, but not all. Setting the prev before the next doesn't make a difference, but we cannot set the values of the neighbouring elements before we set our owns.

Union of two C2L - simplified

Unio(L1: E2*, L2: E2*)

$p := L1 \rightarrow \text{next}; \quad q := L2 \rightarrow \text{next}$

$p \neq L1 \wedge q \neq L2$

$p \rightarrow \text{key} < q \rightarrow \text{key}$	$p \rightarrow \text{key} = q \rightarrow \text{key}$	$p \rightarrow \text{key} > q \rightarrow \text{key}$
$p := p \rightarrow \text{next}$	$s := q$	$s := q$
	$q := q \rightarrow \text{next}$	$q := q \rightarrow \text{next}$
	$L2 \rightarrow \text{next} := q$	$L2 \rightarrow \text{next} := q$
	$q \rightarrow \text{prev} := L2$	$q \rightarrow \text{prev} := L2$
	delete s	$s \rightarrow \text{prev} := p \rightarrow \text{prev}$
	$p := p \rightarrow \text{next}$	$s \rightarrow \text{next} := p$
		$s \rightarrow \text{prev} \rightarrow \text{next} := s$
		$s \rightarrow \text{next} \rightarrow \text{prev} := s$

$q \neq L2$

$e := L2 \rightarrow \text{next}; \quad u := L2 \rightarrow \text{prev}$

$e \rightarrow \text{prev} := L1 \rightarrow \text{prev}; \quad L1 \rightarrow \text{prev} \rightarrow \text{next} := e$

$L1 \rightarrow \text{prev} := u; \quad u \rightarrow \text{next} := L1$

$L2 \rightarrow \text{next} := L2 \rightarrow \text{prev} := L2$

skip

Unio(L1: E2*, L2: E2*)

$p := L1 \rightarrow \text{next}; \quad q := L2 \rightarrow \text{next}$

$p \neq L1 \wedge q \neq L2$

$p \rightarrow \text{key} < q \rightarrow \text{key}$	$p \rightarrow \text{key} = q \rightarrow \text{key}$	$p \rightarrow \text{key} > q \rightarrow \text{key}$
$p := p \rightarrow \text{next}$	$s := q$	$s := q$
	$q := q \rightarrow \text{next}$	$q := q \rightarrow \text{next}$
	unlink(s)	unlink(s)
	delete s	precede(s, p)
	$p := p \rightarrow \text{next}$	

$L2 \rightarrow \text{next} \neq L2$

$s := L2 \rightarrow \text{next}$

unlink(s)

follow(L1 $\rightarrow$ prev, s)

Union and intersection of two C2L

$$H_u \rightarrow [/\text{---}[2]^{q}\text{---}[4]\text{---}[6]\text{---}[/] \leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[1]_r\text{---}[4]\text{---}[5]\text{---}[8]\text{---}[9]\text{---}[/] \leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]\text{---}[2]^{q}\text{---}[4]\text{---}[6]\text{---}[/] \leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_r\text{---}[5]\text{---}[8]\text{---}[9]\text{---}[/] \leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]\text{---}[2]\text{---}[4]^{q}\text{---}[6]\text{---}[/] \leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_r\text{---}[5]\text{---}[8]\text{---}[9]\text{---}[/] \leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]\text{---}[2]\text{---}[4]\text{---}[6]^{q}\text{---}[/] \leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_r\text{---}[5]\text{---}[8]\text{---}[9]\text{---}[/] \leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]\text{---}[2]\text{---}[4]\text{---}[5]\text{---}[6]^{q}\text{---}[/] \leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_r\text{---}[8]\text{---}[9]\text{---}[/] \leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]\text{---}[2]\text{---}[4]\text{---}[5]\text{---}[6]\text{---}[/]^{q} \leftarrow H_u$$

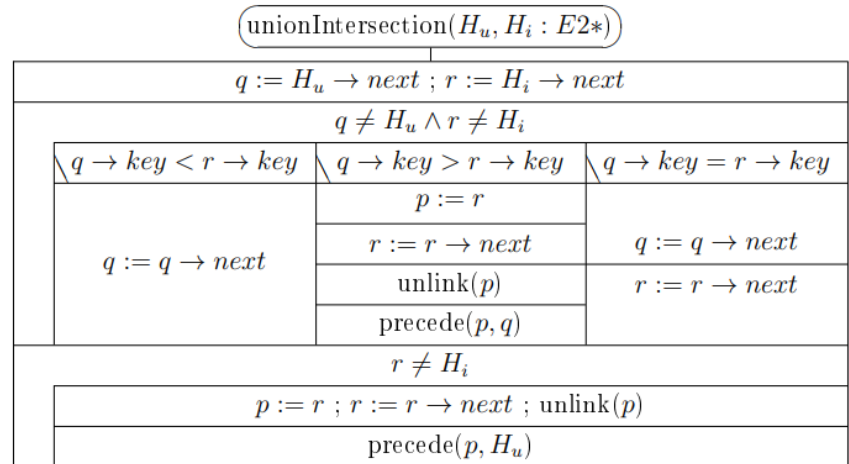
$$H_i \rightarrow [/\text{---}[4]_r\text{---}[8]\text{---}[9]\text{---}[/] \leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]\text{---}[2]\text{---}[4]\text{---}[5]\text{---}[6]\text{---}[8]\text{---}[/]^{q} \leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_r\text{---}[9]\text{---}[/] \leftarrow H_i$$

$$H_u \rightarrow [/\text{---}[1]\text{---}[2]\text{---}[4]\text{---}[5]\text{---}[6]\text{---}[8]\text{---}[9]\text{---}[/]^{q} \leftarrow H_u$$

$$H_i \rightarrow [/\text{---}[4]_r\text{---}[/] \leftarrow H_i$$



Ending

Thank you for your attention!